Cairo University
Faculty of Engineering
Electronics & Communications Dept.

NN ASSINGMENT 1

...

Prepared by:

Abdallah Essam Mohamed Mahmoud	ID: ...
Zeyad Antar Othman Abu-Zaid	ID: 9220331
Abdelrahman Hatem	ID: ...

Submitted to:

Dr. Fadel Dighm

February 2026

Contents

1 Part 1 **2**

2 Part 2: Classical ML Modeling **2**

2.1 Problem Summary and Required Tasks 2

2.2 Implementation Flow 2

2.3 Nonlinear SVM Used (RBF) and Its Specifications 3

2.4 Comparative Results (Accuracy and Processing Time) 3

2.5 Best Results and Confusion Matrices 4

2.6 Conclusions 4

1 Part 1

Part 1 is documented separately.

2 Part 2: Classical ML Modeling

2.1 Problem Summary and Required Tasks

ReducedMNIST is used with 10 classes (digits 0–9), where the training split has 1000 samples per class and the testing split has 200 samples per class.

The required tasks are:

1. Extract three feature families for train and test sets:
   - DCT with 225 dimensions,
   - PCA with cumulative variance at least 95%,
   - HOG features.
2. Train and evaluate:
   - K-means per class with $K = 1, 4, 16, 32$,
   - SVM with linear and nonlinear kernels.
3. Report comparative accuracy and processing time in one final table.
4. Show confusion matrices only for the best K-means result and the best SVM result.
5. Add conclusions.

2.2 Implementation Flow

The implemented flow in Part_2 scripts is:

1. Load ReducedMNIST (JPG folder dataset via KaggleHub, with safe fallbacks).
2. Normalize pixel values to $[0, 1]$.
3. Extract DCT, PCA, and HOG features.
4. Train K-means per class for $K \in \{1, 4, 16, 32\}$ and evaluate on test data.
5. Train SVM with linear and RBF kernels and evaluate on test data.
6. Store predictions, measure real elapsed time, and print summary tables.
7. Visualize confusion matrices and comparative charts.

2.3 Nonlinear SVM Used (RBF) and Its Specifications

The nonlinear model used in this implementation is **C-SVC with RBF kernel** from scikit-learn.

The kernel function is:

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\right)$$

which enables curved (nonlinear) decision boundaries for classes that are not linearly separable.

Exact settings used in code (Part_2/step3_svm.py):

- `kernel='rbf'`
- $C = 10.0$
- $\gamma =$ `scale`
- `decision_function_shape='ovo'` (one-vs-one multiclass)

Feature scaling step before SVM training/inference:

- `StandardScaler()` is fitted on training features.
- The same scaler is applied to both train and test features.

For $\gamma =$ `scale`, scikit-learn uses:

$$\gamma = \frac{1}{n_{\text{features}} \operatorname{Var}(X)}$$

All remaining `SVC` parameters are left at library defaults.

2.4 Comparative Results (Accuracy and Processing Time)

Table 1 summarizes all required combinations with measured real elapsed time.

Classifier	**DCT**		**PCA**		**HOG**	
	Acc. (%)	Time (s)	Acc. (%)	Time (s)	Acc. (%)	Time (s)
K-means $K = 1$	86.85	0.01	86.80	0.00	91.35	0.04
K-means $K = 4$	92.65	2.19	92.30	0.61	93.60	8.00
K-means $K = 16$	94.85	1.50	94.80	1.51	95.00	17.85
K-means $K = 32$	96.15	2.05	95.70	2.14	96.00	29.79
SVM Linear	92.60	1.78	90.65	2.66	96.70	10.80
SVM Nonlinear (RBF, $C = 10$, $\gamma =$scale)	96.90	5.05	95.40	12.54	96.70	31.02

Table 1: Part 2 required comparative table: accuracy and processing time.

2.5 Best Results and Confusion Matrices

Best K-means result: **K-means** $K = 32$ **with DCT** (96.15%).
Best SVM result: **SVM Nonlinear (RBF) with DCT** (96.90%).

Insert confusion matrix image for best K-means result here (K=32 with DCT).

Figure 1: Confusion matrix for the best K-means model.

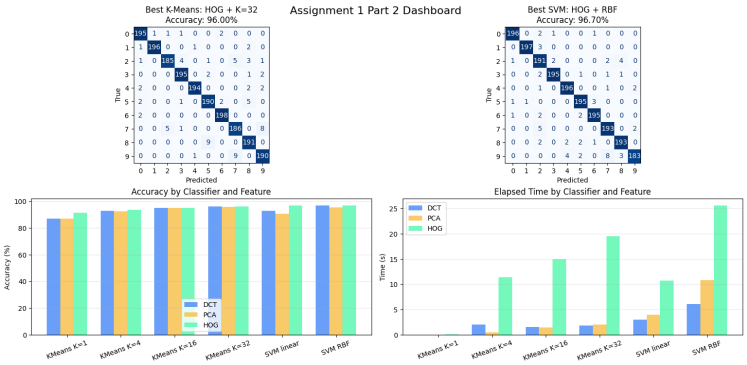

Figure 2: Confusion matrix for the best SVM model.

2.6 Conclusions

1. Increasing K in class-wise K-means consistently improves performance, with best result at $K = 32$.

2. Nonlinear SVM (RBF) gives the highest overall accuracy among all tested combinations.

3. HOG is competitive, especially for linear SVM, but incurs higher runtime.

4. DCT features produced the strongest best-case accuracy for both classifier families in this run.

5. Reporting both accuracy and elapsed time is important because the best-accuracy model is not always the fastest one.